

Homework 2

CSCI 567 — Machine Learning — Fall 2026

Announced Sep 18; due Monday Sep 28, 11:59pm PT, on Gradescope

Logistics. Homework is done in pairs; one submission per pair. Submit two things: (1) a zip file containing your code (both `.py` files) on **Brightspace**; (2) a single PDF write-up on **Gradescope** with your answers to every question, including, for the programming parts, the printed tables and numbers (screenshots of the output are fine), the plots the code produces, and your written explanations. The write-up must be typed in $\text{\LaTeX}$. Each programming part has a few short written questions; answer them in the write-up, with the numbers and plots your code produces. **Total: 50 pts.**

Collaboration and AI policy. You may discuss problems at a high level with other groups, but every pair must write its own solutions and code; do not search for solutions online, share write-ups across groups, or post course material publicly. Late days: 3 total across the semester, at most 1 per assignment (a late day is charged to each partner). AI usage is not allowed for the HWs.

Setup. Use Python 3 (≥ 3.9) with **numpy and matplotlib only**. Starter code and the dataset are provided in the `hw2_code` folder; fill in every function marked `TODO` without changing seeds, splits, step sizes, iteration counts, the λ and γ grids, or the reporting code at the bottom of each file. **The starter code does not handle the bias term for you.** Every model in Problems 1–3 and the linear baseline in Problem 4 must include it.

Notation. As in lecture: labels $y \in \{0, 1\}$, scores $\mathbf{w}^\top \mathbf{x}$ with the bias folded in as a constant-1 last feature, $\sigma(z) = 1/(1 + e^{-z})$, and the mean logistic loss

$$\hat{\mathcal{R}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \left[-y_i \log \sigma(\mathbf{w}^\top \mathbf{x}_i) - (1 - y_i) \log (1 - \sigma(\mathbf{w}^\top \mathbf{x}_i)) \right].$$

Every penalty in this homework leaves the bias coordinate unpenalized.

Problem 1: logistic regression and the ML pipeline (10 pts - Easy)

A real ML workflow is more than a training loop: split the data, tune every knob on a *validation* set, and touch the *test* set only once at the end. Problems 1–3 build that pipeline on one dataset.

The dataset (provided, `data/wdbc.csv`) is the Breast Cancer Wisconsin (Diagnostic) set from the UCI ML Repository: 569 tumors, 30 real-valued features each, label 1 = malignant (37% of examples), 0 = benign. The starter code (`logistic_pipeline.py`) shuffles with a fixed seed and splits 60/20/20 into train/validation/test. Positive class throughout: *malignant*.

- 1.1 (2 pts) Take care of the bias term.** Implement `add_bias` as discussed in the lecture. *Note: Keep the bias in mind for the rest of the homework: think how the bias should be taken care of when implementing regularization.*

1.2 (6 pts) Train it.

- (a) (4 pts) Implement `min_max_scale` (fit the per-feature min/max on the *training* set only, apply the same map to validation and test), `sigmoid`, `logistic_loss`, `gradient`, `train_gd`, `predict_labels`, and `error_rate`, following the numerical hints in the docstrings.
- (b) (1 pt) Train full-batch GD on the scaled features ($\eta = 0.5$, 2000 iterations, $\mathbf{w}_0 = \mathbf{0}$) and report the final training loss, the training and validation error rates, and $\|\mathbf{w}\|_2$ (bias excluded).
- (c) (1 pt) Why would fitting the scaler's min/max on all the data, rather than the training set only, be cheating?

1.3 (2 pts) Break it.

The starter reruns the identical training on the *raw*, unscaled features (some features are $\sim 2000\times$ larger than others).

- (a) (1 pt) Report the final loss and validation error, and include the plot of both loss curves the starter code produces.
- (b) (1 pt) In a sentence or two, say qualitatively what went wrong: what did the large feature scales do to the shape of the loss, and why does the same η that worked in 1.2 misbehave here?

Problem 2: ℓ_2 regularization (10 pts - Easy)

Lecture 03 added $\frac{\lambda}{2} \|\mathbf{w}\|_2^2$ to the least-squares objective (ridge). The same term can be added to any empirical risk; here you add it to logistic regression:

$$\hat{\mathcal{R}}_\lambda(\mathbf{w}) = \hat{\mathcal{R}}(\mathbf{w}) + \frac{\lambda}{2} \sum_{j \neq \text{bias}} w_j^2.$$

2.1 (6 pts) Implement and sweep.

Implement `l2_objective`, `l2_gradient`, and `train_gd_l2`. The starter trains one model per λ in the grid

$$\lambda \in \{0, 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}, 10^{-2}, 3 \cdot 10^{-2}, 10^{-1}, 1\}$$

on the scaled features and prints, for each, the objective, the unpenalized training loss, the training and validation error rates, and $\|\mathbf{w}\|_2$. Include the table and the `reg_errors.pdf` plot in your write-up.

2.2 (4 pts) Read the sweep.

- (a) (2 pts) Write down $\nabla \hat{\mathcal{R}}_\lambda(\mathbf{w})$: Lecture 02's logistic gradient plus one term, with the bias unpenalized.
- (b) (2 pts) Which λ do you pick, and why? In two sentences, explain the shape of the validation error curve as λ grows using Lecture 03's bias-variance tradeoff.

Problem 3: ℓ_1 regularization (10 pts - Medium)

Now swap the penalty for $\lambda \sum_{j \neq \text{bias}} |w_j|$ (Lasso, Lecture 03). The objective is convex but not differentiable at $w_j = 0$, so plain GD does not apply. Use **proximal gradient descent** (ISTA), as in Lecture 03's "Solving Lasso" slides: a gradient step on the smooth part, then the soft-threshold on every non-bias coordinate,

$$\mathbf{v} \leftarrow \mathbf{w} - \eta \nabla \hat{\mathcal{R}}(\mathbf{w}), \quad w_j \leftarrow S_{\eta\lambda}(v_j) = \text{sign}(v_j) \max(|v_j| - \eta\lambda, 0).$$

3.1 (6 pts) Implement and sweep. Implement `soft_threshold` and `train_ista_l1` (same η , iteration count, and λ grid as Problem 2). The starter prints, per λ , the training loss, training and validation error rates, $\|\mathbf{w}\|_2$, and the number of nonzero weights, and saves `sparsity.pdf`. Include the table and plot.

3.2 (4 pts) Read the sweep.

- (a) **(2 pts)** How many of the 30 weights does ridge set *exactly* to zero at $\lambda = 10^{-2}$, versus Lasso? Explain the difference in one sentence from the two update rules.
- (b) **(1 pt)** The starter names the features that survive at $\lambda = 10^{-2}$ and $\lambda = 3 \cdot 10^{-2}$; list them and comment in a sentence on whether the survivors make sense for predicting malignancy.
- (c) **(1 pt)** Pick the sparsest model whose validation error is within one percentage point of the best in either sweep, and state how many features it uses.

Problem 4: kernel logistic regression is a learned nearest-neighbour classifier (20 pts - Hard)

Two classifiers you have not met yet. The simplest classifier there is stores the training set and predicts, for a new $\mathbf{x}$, the label of the closest training point: the **nearest-neighbour** (1-NN) classifier. Its smoothed version, k -NN, takes a majority vote among the k closest training points. A further smoothing weights every training point by how close it is: with $\tilde{y}_i = 2y_i - 1 \in \{-1, +1\}$ and the RBF kernel $\kappa(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$,

$$\text{weighted vote:} \quad s(\mathbf{x}) = \sum_{i=1}^n \kappa(\mathbf{x}, \mathbf{x}_i) \tilde{y}_i, \quad \hat{y}(\mathbf{x}) = \mathbb{1}[s(\mathbf{x}) > 0].$$

None of these is *trained*: the weights are a fixed heuristic (all-or-nothing for k -NN, distance-decaying for the vote). This problem shows that RBF kernel logistic regression is the same kind of classifier, except that the per-point weights are *learned*, and then asks when that matters.

Kernel logistic regression (Lecture 04). Write $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$ for the training kernel matrix. The model is $f(\mathbf{x}) = \sum_i \alpha_i \kappa(\mathbf{x}, \mathbf{x}_i)$, with $\mathbb{P}(y = 1 \mid \mathbf{x}) = \sigma(f(\mathbf{x}))$. It is fit by gradient descent on

$$J(\alpha) = \frac{1}{n} \sum_{i=1}^n \left[-y_i \log \sigma((K\alpha)_i) - (1 - y_i) \log (1 - \sigma((K\alpha)_i)) \right] + \frac{\lambda}{2} \alpha^\top K \alpha,$$

$$\nabla J(\alpha) = K \left[\frac{1}{n} (\sigma(K\alpha) - \mathbf{y}) + \lambda \alpha \right].$$

The starter (`kernel_logistic.py`) generates two 2-D datasets of concentric rings (label 0 near radius 1, label 1 near radius 2, radial noise; 200 training and 500 validation points): a *balanced* one with uniform angles, and an *uneven* one in which class 1 is dense on the right side and sparse on the left while class 0 stays uniform. It imports your Problem 1 functions.

4.1 (7 pts) Implement all three. Implement the kernel model: `rbf_kernel` (no Python loops), `kernel_objective`, `kernel_gradient`, `train_kernel_gd` (GD from $\alpha_0 = \mathbf{0}$; the step size is provided), and `kernel_predict`. Then implement the two heuristics, `knn_predict` and `weighted_vote_predict`. The starter runs everything on the balanced rings ($\lambda = 10^{-3}$, γ from 0.1 to 1000, $k \in \{1, 5, 15\}$) and prints the validation errors; include the table.

Part I: the connection.

4.2 (4 pts) The learned weights are label-signed residuals. Assume K is invertible (true for the RBF kernel on distinct points; take it as given). Let α be a minimizer of J and $p_i = \sigma((K\alpha)_i)$ the fitted probabilities.

- (a) **(1.5 pts)** From $\nabla J(\alpha) = \mathbf{0}$, show that

$$\alpha_i = \frac{y_i - p_i}{n\lambda} \quad \text{for every } i.$$

- (b) **(1.5 pts)** Conclude that $\text{sign}(\alpha_i) = \tilde{y}_i$ and $0 < |\alpha_i| < \frac{1}{n\lambda}$, and that the trained model can be written

$$f(\mathbf{x}) = \sum_{i=1}^n |\alpha_i| \kappa(\mathbf{x}, \mathbf{x}_i) \tilde{y}_i,$$

i.e. a weighted vote of the training labels with distance-decaying weights, passed through a sigmoid — the vote above with per-point multipliers $|\alpha_i|$.

- (c) **(1 pt)** In words: which training points get large weight, which get small weight, and what does λ control?

4.3 (2 pts) The two limits.

- (a) **(1 pt)** As $\gamma \rightarrow \infty$ with everything else fixed, argue that both the weighted vote and kernel logistic regression predict the label of the nearest training point (1-NN). The starter checks this at $\gamma = 1000$; report the two agreement fractions with 1-NN and explain why kernel logistic regression's is the lower one.
- (b) **(1 pt)** As $\gamma \rightarrow 0$, what does each predict? Which quantity in the neighbour heuristics plays the role that γ plays here?

Part II: the difference.

4.4 (2 pts) Balanced rings. Read the table from 4.1 and the figure `balanced_boundaries.pdf` (weighted vote vs. kernel logistic at $\gamma = 1$).

- (a) **(1 pt)** Over what range of γ is each method good?
- (b) **(1 pt)** How do the best errors of k -NN, the vote, and kernel logistic regression compare?

4.5 (5 pts) Uneven density. On the uneven rings the starter prints, for $\gamma \in \{0.3, 1, 3, 10\}$, the validation error of the vote and of kernel logistic regression, overall and on the left half of the plane (where class 1 is sparse), plus k -NN, and saves `uneven_boundaries.pdf`: the two decision regions at $\gamma = 1$ and a third panel of the training points with marker area proportional to $|\alpha_i|$. It also computes $\beta_i = (y_i - p_i)/(n\lambda)$ from your trained model, checks that predicting with β in place of α gives the same labels, and prints the mean $|\beta_i|$ by class and side.

- (a) **(2 pts)** Report the table and the figure. Where does the vote's boundary go wrong, and why, in terms of which class dominates the sum $s(\mathbf{x})$ there?
- (b) **(3 pts)** Report the identity check and the mean weights by class and side. Using 4.2, explain the pattern of the learned weights, and how it repairs the boundary. Why can no fixed choice of γ or k do the same?